

## Ballast Lane Applications — Universidad del Norte

### Internship Program

---

#### Project Overview

Your task is to build a **working web application** in the **language and framework of your choice**. The domain is up to you — a recipe manager, a study planner, a music catalog, a pet adoption board — pick something that interests you.

Your development should be driven by an informal **user story** that you will create and include in your repository.

The application should allow users to create, read, update, and delete records via API endpoints or a user interface. Additionally, the application should support user registration and login, with at least one operation restricted to authenticated users.

**We encourage you to use AI coding tools** (Claude Code, Cursor, GitHub Copilot, Windsurf, or any tool you prefer) to help you build your project. In fact, how you work with AI is a core part of what we evaluate. However — and this is important — **you must understand every line of code in your repository**. During the technical interview, you will be asked to explain your decisions, modify code live, and demonstrate that you truly own the work.

Your submission will be reviewed by our **automated code review pipeline** — the same tools and standards we apply to production projects at Ballast Lane Applications. This pipeline evaluates architecture, code quality, security practices, test coverage, and documentation. It is rigorous. A project that was generated entirely by AI without human judgment will not pass this review, and will not pass the interview.

---

#### Requirements

##### Core Requirements (Everyone)

Regardless of your specialization or preferred stack, every submission must include:

##### 1. A Working Application

- At least one primary entity with full CRUD operations (create, read, update, delete).
- User registration and login (authentication).
- At least one operation that requires authentication and one that does not.
- Persistent data storage (SQL, NoSQL, file-based — your choice, but it must persist across restarts).

##### 2. Code Quality

- Clean project structure with a clear separation of concerns (your API/controllers should not contain database queries directly, your business rules should not live in your UI components, etc.).

# Technical Interview Exercise

---

- Consistent coding style — use a linter and/or formatter appropriate for your language.
- Meaningful names for variables, functions, files, and folders.
- A `.gitignore` that keeps secrets, dependencies, and build artifacts out of the repository.

## 3. Testing

- Write at least **5 meaningful tests** (unit or integration).
- Tests must pass when we run them. Include the command to run them in your README.

## 4. Documentation

- A `README.md` with: what the project does, how to set it up locally, how to run it, and how to run the tests.
- If your project needs seed data or demo credentials, include them and document them.

## 5. Git Practices

- Use meaningful commit messages that describe *what* changed and *why*.
- Your commit history should tell the story of how you built the project — not a single giant commit.
- The repository **must be public** on GitHub.

---

## Choose Your Focus

Depending on your interests and the role you are applying for, go deeper in **at least one** of the following areas. You may combine more than one.

### Backend Focus

- Implement input validation and return appropriate HTTP status codes for errors (400, 401, 404, 409, etc.).
- Add at least one business rule beyond simple CRUD. Examples: users can only edit their own records; items with a past date cannot be modified; a category cannot be deleted if it has associated items.
- Implement pagination or filtering on list endpoints.
- Use environment variables for configuration (database connection, secrets) — never hardcode them.

### Frontend Focus

- Build a responsive UI that works on both mobile and desktop screens.
- Implement client-side form validation with clear, user-friendly error messages.
- Use a component-based architecture (React, Vue, Svelte, Angular, or equivalent).
- Handle loading states and error states gracefully — the user should always know what is happening.

### Infrastructure Focus

- Containerize your application with Docker (Dockerfile and `docker-compose.yml`) so it can be started with a single command.
- Set up a CI pipeline using GitHub Actions that at minimum installs dependencies, runs the linter, and runs the tests.



# Technical Interview Exercise

---

- Add a health check endpoint (e.g., `GET /health`) that returns the application status.
  - Include an architecture diagram or deployment notes in your documentation.
- 

## Agentic Coding Process

**This section is mandatory and carries significant weight in your evaluation.**

We want to see how you think *with* AI, not how AI thinks *for* you.

In your repository, create a file called `AGENTIC.md` that documents the following:

### 1. Tools Used

Which AI coding tools did you use? List them and briefly describe how you used each one.

### 2. Your Approach

How did you structure your work with AI? For example:

- Did you start with a plan and ask AI to implement specific pieces?
- Did you ask AI to scaffold the entire project and then modify it?
- Did you use AI mostly for specific tasks (debugging, writing tests, generating boilerplate)?
- Did you iterate on prompts to refine the output?

### 3. Key Prompts (include 2-3 real examples)

For each prompt, describe:

- What you asked the AI to do.
- A summary or excerpt of what it generated.
- Whether you used it as-is, modified it, or discarded it — and why.

### 4. Critical Evaluation

For at least one significant piece of AI-generated code in your project:

- What did the AI get right?
- What did it get wrong, or what did you improve?
- How did you verify that the code actually works correctly?
- Did the AI introduce any security issues, bugs, or bad practices that you had to fix?

### 5. What You Learned

What concepts or techniques did you learn during this process that you did not know before starting the project? Be specific.

---

## What We Are NOT Looking For

To set clear expectations:

- A massive, over-engineered project that you cannot explain.
- Copy-pasting AI output without understanding what it does.
- Perfect-looking code paired with zero understanding of the underlying concepts.
- Skipping the `AGENTIC.md` documentation.

# Technical Interview Exercise

---

- A project that does not run.
  - A single commit with the message “initial commit” containing the entire project.
- 

## Deliverable

Submit the URL of your **public GitHub repository**.

The repository must contain:

- Your application source code.
  - A `README.md` with setup and run instructions.
  - An `AGENTIC.md` documenting your AI-assisted development process.
  - Working tests with instructions on how to run them.
- 

## Presentation and Code Review

You will present your project to a technical panel via Google Meet or Zoom (20-30 minutes total). During the session, you will screen-share your GitHub repository and/or your IDE. Be prepared to:

### 1. Demo (3-5 minutes)

Show your working application. Walk through the main user flows: creating a record, logging in, performing an authenticated operation.

### 2. Architecture Walkthrough (5-7 minutes)

Explain your project structure, your design decisions, and how the different layers of your application interact. Why did you choose this stack? How did you organize your code?

### 3. Agentic Process Discussion (5 minutes)

Walk us through your `AGENTIC.md`. Show us specific examples of how you used AI, what you accepted, what you rejected, and how you validated the output.

### 4. Live Code Review (5-10 minutes)

The panel will select sections of your code and ask you to:

- Explain what the code does and why you wrote it that way.
  - Identify what you would improve if you had more time.
  - Make a small modification on the spot (e.g., add a validation rule, fix a bug we introduce, change a query).
- 

## Evaluation Criteria

Your project will be evaluated on the following criteria:

### Functionality (20%)

The application works as described. CRUD operations, authentication, and data persistence all function correctly without errors.

# Technical Interview Exercise

---

## **Code Quality (20%)**

Clean project structure, readable code, consistent style, proper separation of concerns. The code should look like something you would be comfortable maintaining six months from now.

## **Agentic Process (20%)**

Thoughtful and honest documentation of your AI-assisted development process. We value critical thinking over volume — showing us one prompt you improved is worth more than ten prompts you blindly accepted.

## **Understanding (15%)**

You can explain your code, answer questions about your architectural decisions, and make modifications during the live review. This is where we distinguish between someone who built a project and someone who watched AI build a project.

## **Testing (15%)**

Your tests are meaningful (not trivial assertions), they pass, and they actually validate the behavior described in your requirements.

## **Documentation (10%)**

Clear README with setup instructions. Complete AGENTIC.md. A developer unfamiliar with your project should be able to clone it, set it up, and run it by following your documentation.

---

*Good luck. We are excited to see what you build.*